

Files and Stuff

Vassilis Markos

December 9, 2024

E1. Consider the following python script:

```
1 # test.py
2 file = open("test.txt", "w")
3 file.write("Hello, World!")
4 file.close()
5
```

- (a) What is the purpose of the above script?
- (b) Where is `test.txt` located?
- (c) Using the native `os` python module, try to save the above file at `./my_dir/another_test.txt`.
- (d) Assume you have the following directory structure in your project:

```
parent_dir
|
|---working_dir
|   |---test.py
|   |---test.txt
```

Extend the above python script so as to achieve the following directory structure:

```
parent_dir
|
|---working_dir
|   |---test.py
|   |---test.txt
|---sibling_dir
|   |---copy_test.txt
```

Essentially, you have to create a new directory, named `sibling_dir`, and then save a copy of `test.txt` in `sibling_dir`. Both the directory and the file should be created using **exclusively python** (you should not use your OS capabilities).

- (e) Write the script of question E1(d) using `with` statements.

E2. The Morse code has been used to communicate using just two symbols: dot (·) and dash (—). All basic symbols, i.e., capital case letters of the Latin alphabet (A-Z) and decimal digits (0-9) are represented using just dashes and dots. You can read more on Morse code in the following Wikipedia lemma:
https://en.wikipedia.org/wiki/Morse_code.

- (a) Write a python function named `text_to_morse()` which accepts a string consisting of alphanumeric characters, digits, and, potentially, spaces, and translates it to Morse code. Use " " (the space character) as a word delimiter in your Morse translation and | as a character delimiter.
- (b) Write a python function named `morse_to_text()` which accepts a string consisting of dashes and dots (i.e., Morse code) and the word and character delimiters used above and returns the corresponding text. In case you find any invalid combinations of dashes and dots, your function should raise an appropriate exception.
- (c) Consider the following piece of Morse code:

..---

This can be interpreted as “2” (a single character), as well as “UM” (i.e., as two characters). Such ambiguities make the use of a character delimiter necessary. Assume that you receive a Morse message in which characters have not been delimited, thus there are more than one possible interpretations, as above.

- i. How would you address this problem? Describe your idea in text (e.g., using pseudocode).
 - ii. Write a python function, `decipher_morse()`, that accepts a Morse code string consisting only of dashes and dots and returns a list containing all possible interpretations of that string.
- (d) Write a python script that:
 - asks the user whether they wish to translate from natural language to Morse code or vice versa;
 - asks the user to provide a file to be translated, either in natural language or Morse code (depending on user’s previous choice);
 - translates the provided piece of text / Morse code and stores the output into an appropriately named file.

In your solution, make use of any functions developed above and use exceptions when appropriate. Moreover, consider the case where the user provides Morse code that has not been character

delimited (i.e., they have forgotten to use ”—”). In that case, inform the user about that and provide all possible interpretations, asking them to choose which one is the appropriate one.

E3. Your local school has asked you to compute some key descriptive statistics about its students’ height distribution. Namely, they have asked you to compute:

- mean height,
- standard height deviation,
- median height (50% percentile),
- Q1 and Q3 (25% and 75% percentiles).

To do so:

- (a) Write a python function, `read_input()`, that repeatedly asks the user for numbers (student heights) and returns a list with these numbers.
- (b) Write a python function, `mean_value()`, which accepts a list of numbers as an argument and returns its mean value.
- (c) Write a python function, `std()`, which accepts a list of numbers as an argument and returns its standard deviation.
- (d) Write a python function, `percentile()`, which accepts a list of numbers and a number, p , between 0.0 and 1.0 as arguments and returns the corresponding p percentile.
- (e) Write a python function, `write_results()`, which accepts a list of numbers and computes its length, its mean value, standard deviation, median, Q1 and Q3 and writes them in a file named `results.txt`. This function should check for a file with the same name and, in case it exists, it should save it as `results_xxx.txt`, where `xxx` is the number of files with the same name already found. For instance, if there has already been saved a file named `results.txt`, you should save your file as `results_001.txt`. If there are 3 files, named `results.txt`, `results_001.txt` and `results_002.txt`, you should save your file as `results_003.txt` and so on.

At last, the format of your file should be as follows:

----- -----
size 200
mean 168.06
std 28.75
median 167
Q1 158

Q3	179
-----	-----

Hint: If you have any questions on any statistics terms used above, have a look at a really (really) brief crash-course on descriptive statistics at the end of this file.

- E4. (a) Write a function, `write_list()`, that accepts a list of integers and a string as arguments, and writes this list into a `.txt` file named according to the provided string. For instance, if provided with the following list:

```
a = [3, 5, 1, 0, -2]
```

and the string `"numbers"`, `write_list()` should store this list as the following string:

```
3,5,1,0,-2
```

in a file named `numbers.txt`. Take care to handle the case of a file with the same name already existing at the same directory in a way similar to E3(e).

- (b) Write a function, `read_list()`, that accepts a filename of a file containing a list (as a string), reads the corresponding file and returns the list included in it as a python list. For instance, in case the string `"numbers.txt"` is provided, `read_list()` should open up the file `numbers.txt` and read its contents, e.g.:

```
3,5,1,0,-2
```

Then, it should return the following list:

```
read_list() -> [3, 5, 1, 0, -2]
```

- (c) Write a python script that:

- Reads a list of integers from a user (input terminates by entering "q" instead of a positive integer).
- Saves that list into a file (asking the user for the desired file name).
- Computes the mean value, standard deviation and median of the provided numbers and prints them on screen.
- Asks the user if they wish to enter some more numbers:
 - If no, the program terminates.
 - If yes, the program: (i) reads the list from the saved file, (ii) then reads any numbers the user has to provide, (iii) appends them to the loaded list, (iv) recomputes the corresponding mean value, (v) standard deviation and median, (vi) and then stores the new list at the same

file, (vii) and asks again if the user wants to provide some more numbers.

A Crash-course in Descriptive Statistics

Descriptive statistics is that sub-field of statistics concerned with the description of a given sample, providing a draft overview of its distribution through statistical measures. Some of the key statistical measures include mean value, standard deviation, and 25%, 50% and 75% percentiles (Q1, Q2 / median, Q3).

Measures of Position and Dispersion

For instance, consider the following sample of 7 observations:

$$6, 4, 7, 8, 1, 0, 4.$$

The size of this sample is $n = 7$ (the count of the observations), while its mean value is computed by summing all observations and dividing by sample size:

$$\mu = \frac{6 + 4 + 7 + 8 + 1 + 0 + 4}{7} = \frac{30}{7} \approx 4.29.$$

To compute the standard deviation of this sample, we use the following table:

x_i	6	4	7	8	1	0	4
$x_i - \mu$	-1.71	-0.29	2.71	3.71	-3.29	-4.29	-0.29
$(x_i - \mu)^2$	2.92	0.08	7.34	13.76	10.82	18.40	0.08
$\sum (x_i - \mu)^2$	53.4						
$s^2 = \frac{1}{n} \sum (x_i - \mu)^2$	$\frac{53.4}{7} \approx 7.63$						
$s = \sqrt{s^2}$	$\sqrt{7.63} \approx 2.76$						

Table 1: A visual way to compute a sample's standard deviation, $s = \sqrt{s^2}$. x_i denotes the i -th observation.

In a nutshell, to compute $s = \sqrt{s^2}$ we work as follows:

- We subtract the mean value, μ , from each sample observation, x_i .
- We square all $x_i - \mu$ computed above.
- We sum all squares computed above.
- We divide the computed sum by sample size, n , which yields s^2 .
- We take the square root of the above, obtaining s .

Percentiles

In general, a p percentile is that observation (or number, in general) that leaves $p\%$ of the sample below it and the rest $(100 - p)\%$ of the sample above it (assuming that the sample is ordered). When it comes to computing percentiles, we first have to sort our samples. In our case, this means that from:

$$6, 4, 7, 8, 1, 0, 4,$$

we get:

$$0, 1, 4, 4, 6, 7, 8.$$

Then, the median (Q2, 50% percentile) is easily computed by finding the median observation, i.e. the fourth observation out of the 7 in our sample. Thus, in our case, $Q2 = 4$. In general, the median observation is found at position $\frac{n+1}{2}$, which, in our case is $\frac{7+1}{2} = 4$, so, the fourth observation, as claimed above. In a similar fashion, Q1 is found at position $\frac{n+1}{4} = \frac{7+1}{4} = 2$, so $Q1 = 1$ in our case, and $Q3$ is found at position $\frac{3(n+1)}{4} = \frac{3(7+1)}{4} = 6$, so $Q3 = 7$. Overall:

- Q1 is found at position: $\frac{n+1}{4}$.
- Q2 is found at position: $\frac{n+1}{2}$.
- Q3 is found at position: $\frac{3(n+1)}{4}$.

Consider the following sample:

$$5, 6, 1, 4, 5, 3.$$

Sorted, this reads:

$$1, 3, 4, 5, 5, 6.$$

To compute the median, we have to look at position:

$$\frac{n+1}{2} = \frac{6+1}{2} = 3.5.$$

But, where is position 3.5 in the above sample? In such cases, where a sample has an even size ($n = 6$ in our case), we have to look at the two positions closest to 3.5, i.e., 3 and 4 in our case. At those positions we have 4 and 5, respectively, so Q2 in our case is:

$$Q2 = \frac{4+5}{2} = \frac{9}{2} = 4.5.$$

Similarly, for Q1 we have to look at position:

$$\frac{n+1}{4} = \frac{7}{4} = 1.75.$$

This means that we should take the average of observations at positions 1 and 2, i.e.:

$$Q1 = \frac{1+3}{2} = 2.$$

At last, for Q3 we look at position:

$$\frac{3(n+1)}{4} = \frac{3 \cdot 7}{4} = 5.25,$$

so, at positions 5 and 6:

$$Q3 = \frac{5+6}{2} = \frac{11}{2} = 5.5.$$